

Supplement: variational aggregation objective and weight control

Supplement: variational aggregation objective and weight control

This supplement gives the full derivation behind sec. 5.8.2: the server-side aggregator `robust_aggregate` is a heuristic,

Supplement: variational... (part 2)

and a single change of divergence direction turns it into block-coordinate descent on a stated free energy with a derived, redescending effective-weight update.

Supplement: variational... (part 3)

We work throughout with categorical local posteriors $q_n(s)$ over the shared latent factor, base weights $w_n > 0$, robustness $c > 0$, and the consensus q on the probability simplex. Let $\lambda > 0$ be the entropy-weight coefficient, with the current default at $\lambda = 1.0$.

Why the sharp heuristic is not yet variational

The sharp server rule is empirically strong, but this repository has not established a variational certificate for it.

Why the sharp heuristic is... (part 2)

The heuristic of eq. 10 alternates a *reverse-KL* weight update $a_n \leftarrow w_n \exp(-c \text{KL}(q_n \parallel q))$.

Why the sharp heuristic is... (part 3)

It couples that update with the log-linear consensus $q \leftarrow \text{softmax}(\sum_n a_n \log q_n)$. For the natural direct objective

$\sum_n a_n \text{KL}(q_n \parallel q) + \frac{1}{c} \text{KL}_{\text{gen}}(a \parallel w)$, the reverse-KL rule is the a -minimizer.

Why the sharp heuristic is... (part 4)

The q -minimizer is the *arithmetic* (linear) pool $q \propto \sum_n a_n q_n$, not the log-linear pool. The executable orientation witness confirms this finite-simplex mismatch. The following proposition goes further, while retaining a deliberately narrow scope.

Why the sharp heuristic is... (part 5)

$$Q(a; s) = \text{softmax} \left(\sum_n a_n \log s_n \right) \quad (24)$$

Why the sharp heuristic is... (part 6)

The proposed raw q -block is the map in eq. 24.

Why the sharp heuristic is... (part 7)

$$F(q, a; s, w) = \sum_n a_n \text{KL}(q \parallel s_n) + R(a, w) + G(q). \quad (25)$$

Why the sharp heuristic is... (part 8)

Proposition (Scoped separable raw-log-pool no-go)

For $K \geq 2$, no objective in (25) satisfying the declared separability and differentiability conditions has $Q(a; s)$ as its q -coordinate minimizer for every positive interior a and s .

Why the sharp heuristic is... (part 9)

Precisely, G is continuously differentiable and independent of a, s , while R is independent of q, s . Under those conditions, this objective class cannot realize both block maps of the implemented raw-weight heuristic.

Why the sharp heuristic is... (part 10)

Proof sketch. Fix any non-uniform interior q , a positive scalar α , and one local posterior constructed in eq. 26:

Why the sharp heuristic is... (part 11)

$$s_i^{(\alpha)} = \frac{q_i^{1/\alpha}}{\sum_j q_j^{1/\alpha}}. \quad (26)$$

Why the sharp heuristic is... (part 12)

Then $Q(\alpha; s^{(\alpha)}) = q$. Writing Π for projection onto the tangent space of the simplex, first-order stationarity of eq. 25 at that same q requires $\Pi[(\alpha - 1) \log q + \nabla G(q)] = 0$.

Why the sharp heuristic is... (part 13)

The unit-scale construction forces $\Pi \nabla G(q) = 0$; any different positive scale then forces $\Pi \log q = 0$, contradicting the non-uniform choice of q . The executable witness records both exact log-pool identities and the nonzero tangential contradiction.

Why the sharp heuristic is... (part 14)

A companion witness also blocks the obvious normalized-weight escape within the same natural data-term class: two interior consensuses yield the same normalized reverse-KL weights but different forward-KL data-term differences,

Why the sharp heuristic is... (part 15)

so a q -independent differentiable $R(a, w)$ cannot satisfy both simplex-stationarity equations. The implementation itself uses raw effective weights, so this companion is a scope check rather than a description of the production update.

Why the sharp heuristic is... (part 16)

Artifact	Implementation surface and declared scope
Raw contradiction	<code>server_theory.py</code> ; interior inputs and raw q -block

Why the sharp heuristic is... (part 17)

The witness table in tbl. 13 records the deterministic implementation surfaces that bind this scoped result to the typed analysis report.

Why the sharp heuristic is... (part 18)

The proposition does **not** say that no objective of any kind exists. It does not exclude nonseparable q - a couplings, source-dependent terms, non-differentiable constructions, or objectives that encode selected fixed points without reproducing the update blocks for all interior inputs.

Why the sharp heuristic is... (part 19)

Thus sec. 5.8 retains the heuristic label and claims only the recovery limit, the scoped negative result above, and conditional empirical behavior — never a bounded-influence property or an objective-backed status.

Aggregation free energy and its block minimizers

Definition (Aggregation free energy)

For $c, \lambda > 0$, consensus q , and nonnegative effective weights $a = (a_n)$, define $F_\lambda(q, a)$ by (12), where $\text{CE}(q, q_n) = -\sum_i q_i \log q_{n,i}$, $H(q)$ is entropy, and $\text{KL}_{\text{gen}}(a \| w) = \sum_n g_n$ for $g_n = a_n \log(a_n/w_n) - a_n + w_n$.

Aggregation free energy... (part 2)

The q -block. For $\lambda > 0$, fixing a , the q -dependent part of F_λ is denoted $J(q; a)$ and can be written in either of two equivalent forms:

Aggregation free energy... (part 3)

$$\begin{aligned} J(q; a) &= \sum_n a_n \text{CE}(q, q_n) - \lambda H(q), \\ J(q; a) &= \lambda \sum_i q_i \log q_i \\ &\quad - \sum_i \sum_n q_i a_n \log q_{n,i}. \end{aligned} \tag{27}$$

Aggregation free energy... (part 4)

Adding a Lagrange multiplier to the block in eq. 27 for $\sum_i q_i = 1$ and differentiating gives $\lambda \log q_i + \lambda - \sum_n a_n \log q_{n,i} + \mu = 0$, i.e.

Aggregation free energy... (part 5)

$$\begin{aligned} q_i &\propto \exp\left(\frac{1}{\lambda} \sum_n a_n \log q_{n,i}\right), \\ q_i &= \text{softmax}\left(\frac{1}{\lambda} \sum_n a_n \log q_n\right)_i. \end{aligned} \tag{28}$$

Aggregation free energy... (part 6)

the product of the weighted experts — the consensus update of eq. 13. At the default $\lambda = 1.0$, the $-H(q)$ term sharpens the weighted geometric mean into the product-of-experts (the entropy bonus that makes the project's log-linear pool a product rather than a geometric average).

Aggregation free energy... (part 7)

The a -block. Fixing q ,
 $\partial F / \partial a_n = \text{CE}(q, q_n) + \frac{1}{c} \log(a_n/w_n) = 0$, so

Aggregation free energy... (part 8)

$$a_n = w_n \exp(-c \text{CE}(q, q_n)), \quad (29)$$

Aggregation free energy... (part 9)

the weight update of eq. 13. Because $\text{CE}(q, q_n) = H(q) + \text{KL}(q \parallel q_n)$, the forward direction $\text{KL}(q \parallel q_n)$ — not the heuristic's reverse $\text{KL}(q_n \parallel q)$ — is the one consistent with the consensus update.

Aggregation free energy... (part 10)

Each block update is the *exact* minimizer of its block, so alternating them is block-coordinate descent: F is non-increasing at every half-step. When the iterates converge, their fixed point is coordinatewise stationary.

Aggregation free energy... (part 11)

The implementation keeps numerical failure handling outside that theorem. If finite-precision underflow collapses all effective weights, it records a fallback event, substitutes the declared base weights to return a valid probability vector, and does not certify the substituted trajectory as converged.

Aggregation free energy... (part 12)

Such a trace is diagnostic evidence about the solver boundary, not an instance of the exact block-descent result.

Conservative-server properties

Theorem (Properties)

For $c, \lambda > 0$, (28)–(29) make F non-increasing. At convergence, (q^, a^*) is coordinatewise stationary. As $c \rightarrow 0$, $a_n \rightarrow w_n$ and q tends to the tempered log-linear pool; $\lambda = 1.0$ gives (9). Finally, $a_n = w_n \exp[-c \text{CE}(q, q_n)] \leq w_n$, and $\text{KL}(q \parallel q_n) \rightarrow \infty$ gives $a_n \rightarrow 0$.*

Conservative-server... (part 2)

The variational aggregator therefore shares the project log-linear-pool corner of (10). Under the qualified bridge of sec. 5.8, this is only the categorical message-combination specialization, not the complete source protocol.

Conservative-server... (part 3)

The final inequality makes the raw effective-weight update bounded and redescending relative to the realized consensus. It does not by itself establish a bounded influence function or finite gross-error sensitivity for the normalized consensus estimator.

Conservative-server... (part 4)

The objective F is biconvex (each block convex, the coupling $\sum_n a_n \text{CE}(q, q_n)$ bilinear), so the result concerns monotone block updates and converged coordinatewise fixed points, not guaranteed convergence to or certification of a global minimum.

Conservative-server... (part 5)

The effective-weight regime, and why multi-start matters. The weight bound $a_n \leq w_n$ is unconditional ($\text{CE}(q, q_n) \geq 0$ always). The *collapse* $a_n \rightarrow 0$ is driven by the agent's divergence *from the realized consensus* $\text{KL}(q \parallel q_n)$, and the consensus itself depends on the weights.

Conservative-server... (part 6)

Because F is biconvex, this couples into a subtlety an adversarial review of this work surfaced: a *near-one-hot* saboteur (contamination rate $\rightarrow 1$) already captures the product-of-experts, so a descent seeded *at* that pool stays in a consensus-capture basin (high F) where the saboteur keeps its weight —

Conservative-server... (part 7)

even against an honest majority.

Conservative-server... (part 8)

The repair is to search the stated objective more carefully: `variational_aggregate` runs **multi-start** block-coordinate descent (the pool, the uniform belief, and the arithmetic-mean seeds) and returns the lowest-observed- F converged candidate.

Conservative-server... (part 9)

In the configured colony, the uniform/arithmetic seeds reach a lower- F *vetoing* basin, so the saboteur is suppressed even at the simplex vertex (pinned qualitatively by the near-vertex multi-start test) — the $267.1\times$ suppression of fig. 17 is measured across the swept contamination grid,

Conservative-server... (part 10)

whose most extreme point sits just below rate 1.

Conservative-server... (part 11)

What remains fundamental to *every* robust fusion rule, and is not claimed away: with no honest majority — a colony split with no anchoring plurality — there is no truth to recover.

Conservative-server... (part 12)

The observed suppression is conditional on the tested colonies and the fixed point selected by a finite multi-start heuristic.

Conservative-server... (part 13)

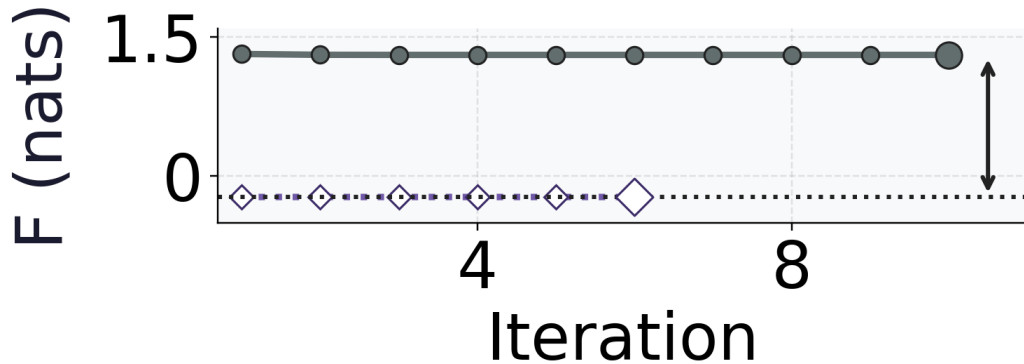
fig. 21 makes the capture and the escape concrete on a near-vertex colony: the single (log-linear-pool) start settles at $F = 1.3092$ (the higher observed basin, where the saboteur keeps its weight), while one of the configured alternative starts reaches a lower observed basin at $F = -0.2305$ —

Conservative-server... (part 14)

a gap of 1.5397 nats. This finite multistart comparison shows that the natural seed can be captured; it neither exhausts all basins nor certifies a global optimum.

Conservative-server... (part 15)

Configured initializations



Conservative-server... (part 16)

Filled circles / solid: single start. Open diamonds / dotted: multistart. Both use the same variational method.

Conservative-server... (part 17)

Single-start final F: 1.31 nats
Multi-start final F: -0.231 nats
Basin gap ΔF : 1.54 nats

Conservative-server... (part 18)

Iterations: single=10, multi=6

Conservative-server... (part 19)

Two deterministic traces from one configured colony. No uncertainty interval applies. Observed basins do not certify a global optimum.

Numerical witnesses for descent and influence bounds

The analysis pipeline runs `variational_aggregate` at robustness $c = 1.50$ on a contaminated colony and records the free energy after each iteration.

Numerical witnesses for... (part 2)

The descent falls from $F = 3.2458$ to $F = 2.3780$ (a monotone drop of 0.8678 over 11 iterations, converged: Yes); the largest single-step *increase* is 8.88×10^{-16} , machine zero — the monotonicity of the theorem,

Numerical witnesses for... (part 3)

witnessed numerically and drawn in fig. 16.

Numerical witnesses for... (part 4)

For the effective-weight diagnostic, one agent is drifted from healthy toward a confident-wrong delta and its normalized influence is read at each drift. Clean, it carries 0.143 of the pool; at the most extreme swept drift it carries below 0.001 — a factor of 267.1 (computed from the unrounded influences,

Numerical witnesses for... (part 5)

not the display-rounded values above) below the fixed 0.143 the naive pool would still grant it (fig. 17).

This makes the redescending normalized-weight behavior visible on the tested path; it is not an estimator-level B-robustness proof.

Tempered aggregation family for the accuracy-guarantee trade

The aggregator of sec. 12.2 fixes the entropy term at unit weight. Relaxing that single coefficient generates a one-parameter *tempered* family. Introduce an entropy weight $\lambda > 0$ —

Tempered aggregation... (part 2)

the **inverse temperature** is $1/\lambda$ — and minimize

Tempered aggregation... (part 3)

$$F_{\lambda}(q, a) = \sum_n a_n \text{CE}(q, q_n) - \lambda H(q) + \frac{1}{c} \text{KL}_{\text{gen}}(a \parallel w). \quad (30)$$

Tempered aggregation... (part 4)

Repeating the q -block derivation of eq. 28 with the entropy scaled by λ leaves the a -block **untouched** and tempers only the consensus update:

Tempered aggregation... (part 5)

$$\begin{aligned} q &\propto \exp\left(\frac{1}{\lambda} \sum_n a_n \log q_n\right), \\ a_n &= w_n \exp(-c \text{CE}(q, q_n)). \end{aligned} \tag{31}$$

Tempered aggregation... (part 6)

The $\lambda \downarrow 0$ endpoint is separately implemented as a deterministic tied-argmax rule; it is not obtained by substituting $\lambda = 0$ into eq. 30 or eq. 31.

Tempered aggregation... (part 7)

The weight update is **independent of** λ : the bound $a_n \leq w_n$ with collapse $a_n \rightarrow 0$ as $\text{KL}(q \parallel q_n) \rightarrow \infty$ is unchanged, so the **raw effective-weight bound of sec. 12.3 holds for every** $\lambda > 0$.

Tempered aggregation... (part 8)

At $\lambda = 1.0$ the temperature is unity and eq. 31 is **identical** to the current axis-3 aggregator eq. 13 — the default is bit-identical, not merely close.

Tempered aggregation... (part 9)

The $c \rightarrow 0$ recovery of sec. 12.3 generalizes to the *tempered* log-linear pool

$q \propto \exp(\frac{1}{\lambda} \sum_n w_n \log q_n)$; at $\lambda = 1.0$ this is exactly $\text{softmax}(\sum_n w_n \log q_n)$ — the project's **log-linear pool** eq. 9.

Tempered aggregation... (part 10)

Under the shared-support, posterior-log-potential, and fixed-weight assumptions of sec. 5.8, that pool is a categorical specialization of Friston Eq. 7's message-combination term, not a reconstruction of the complete source protocol.

Tempered aggregation... (part 11)

Positive-temperature members away from that default are tempered pools, not Friston Eq. 7 itself; the project recovery checks, including ISC-10, remain project-local.

Tempered aggregation... (part 12)

A small empirical sweep over $\lambda \in \{0.1, 0.2, 0.3, 0.5, 0.7, 1\}$ on 10 contaminated colonies (5 agents, 2 adversarial) asks whether a single λ^* makes the conservative aggregator narrow the gap to the sharp `robust_aggregate` point-accuracy.

Tempered aggregation... (part 13)

The closest observed weight is $\lambda^* = 0.3$ with an accuracy gap of 0.0008. **A λ^* narrows the tested accuracy gap while preserving the stated weight bound on this grid.** If no λ closes that gap while preserving the derived weight update, the result is the conservatism trade-off of sec. 8.13,

Tempered aggregation... (part 14)

not a defect to hide.